

## Señales

Una señal es un “aviso” que puede enviar un proceso a otro proceso. El sistema operativo Unix se encarga de que el proceso que recibe la señal la trate inmediatamente. De hecho, termina la línea de código que está ejecutando y salta a la función de tratamiento de señales adecuada. Cuando termina de ejecutar esa función de tratamiento de señales, continúa con la ejecución en la línea de código donde lo había dibujado.

El sistema operativo envía señales a los procesos en determinadas circunstancias. Por ejemplo, si en el programa que se está ejecutando en una shell se aprieta **Ctrl-C**, se está enviando una señal de terminación al proceso. Este la trata inmediatamente y sale. Si el programa intenta acceder a una memoria no válida (por ejemplo, accediendo al contenido de un puntero a **NULL**), el sistema operativo detecta esta circunstancia y le envía una señal de terminación inmediata, con lo que el programa “se cae”.

**Todas las señales pueden ser ignoradas o bloqueadas, a excepción de SIGSTOP y SIGKILL, que son imposibles de ignorar.**

Una limitación importante de las señales es que no tienen prioridades relativas; es decir, si dos señales llegan al mismo tiempo a un proceso, puede que sean tratadas en cualquier orden, no es posible asegurar la prioridad de una en concreto. Otra limitación es la imposibilidad de tratar múltiples señales iguales: si llegan 14 señales SIGCONT a la vez, por ejemplo, el proceso funcionará como si hubiera recibido sólo una.

Cuando se quiere que un proceso espere a que le llegue una señal, se usará la función `pause()`. Esta función provoca que el proceso (o thread) en cuestión “duerma” hasta que le llegue una señal. Para capturar esa señal, el proceso deberá haber establecido un tratamiento de la misma con la función `signal()`.

La función `pause()` no recibe ningún parámetro y retorna `-1` cuando la llamada a la función que captura la señal ha terminado. La función `signal()` tiene un poco más de miga: recibe dos parámetros, el número de señal que se desea capturar (los números en el sistema en concreto en el que se está se pueden obtener ejecutando “kill -l” y un puntero a una función que se encargará de tratar la señal especificada.

**Ejemplo 2 de signal:**

```

#include <signal.h>
#include <stdio.h>
#include <stdlib.h>

//Declaramos los prototipos de funciones
void manejador(int signum);

//Variables globales
int con = 0;

//Función principal
main(int argc, char **argv)
{

    //Capturamos la señal SIGINT
    signal(SIGINT, manejador);

    printf("Ejemplo de signal\n");

    //Hacemos un bucle para permitir que se hagan hasta 5 SIGINTs
    while (con <= 5)
        ;
}

//Funcion manejador
//Cuando se presiona un sigint se llama a esta función
void manejador(int signum)
{
    printf("\nRecibi la señal sigint\n");
    con++;
    signal(SIGINT, manejador);
}

```

Como se puede observar, capturar una señal es bastante sencillo. Intentar ahora ser los emisores de señales a otros procesos. Si se quiere enviar una señal desde la línea de comandos, se utilizará el comando “kill”. La función de C que hace la misma labor se llama, originalmente, kill(). Esta función puede enviar cualquier señal a cualquier proceso, siempre y cuando se tengan los permisos adecuados (las credenciales de cada proceso, explicadas anteriormente, entran ahora en juego (uid, euid, etc.)).

Su prototipo es el siguiente:

```
intkill(pid_tpid, intsig);
```

### Ejemplo de kill:

```
#include <stdio.h>
#include <signal.h>
#include <sys/types.h>

//Declaramos los prototipos de funciones
void manejador(int signum);

//Variable global
int bandera = 1;

//Función principal
main(int argc, char **argv)
{
    //Declaramos variables
    int status, pid;

    //Si fork() es igual a 0 entonces es hijo
    if ((pid = fork()) == 0)
    {
        //Proceso hijo
        printf("Soy hijo y estoy esperando una señal de mi padre, mi pid es:
%d\n", getpid());
        //Capturamos la señal SIGUSR1
        signal(SIGUSR1, manejador);
        //Utilizamos la variable bandera para que el proceso no termine
        while (bandera)
            ;
        //Usamos el comando kill para enviar una señal
        //Se necesita el pid del padre y la señal
        kill(getppid(), SIGUSR2);
    }
    else
    {
        //Proceso padre
        //Capturamos la señal SIGUSR2
        signal(SIGUSR2, manejador);
        printf("Soy Padre, mi pid es: %d\n", getpid());
        //Esperamos 3 segundos
        sleep(3);
        //Usamos el comando kill para enviar una señal
        //Se necesita el pid y la señal
        kill(pid, SIGUSR1);
        //Esperamos que termine el hijo
    }
}
```

```

        wait(&status);
        printf("Mi hijo termino con un estado: %d\n", status);
    }
}

//Funcion manejador
void manejador(int signum)
{
    //Si la señal es SIGUSR1 entonces
    if (signum == SIGUSR1)
    {
        printf("Recibi una senal de mi padre %d\n", signum);
    }
    //Si es SIGUSR2 entonces
    else
    {
        printf("Recibi una senal de mi hijo %d\n", signum);
    }
    bandera = 0;
}

```

Ejemplo 2 de kill:

```

//El padre debe mandar una señal SIGUSR1 O SIGUSR2 al hijo
//La señal debe pasarse a través de la consola
#include <stdio.h>
#include <signal.h>
#include <sys/types.h>

//Prototipos
void manejador(int signum);
void manejador2(int signum);

//Variables globales
int bandera = 1;
int pid, status;
char *parametro;

main(int argc, char **argv)
{
    //Guardamos el argumento 1 en la variable parametro
    parametro = argv[1];
    //Capturamos las señales
    signal(SIGUSR1, manejador2);
    signal(SIGUSR2, manejador2);
}

```

```

if ((pid = fork()) == 0)
{
    // proceso hijo
    printf("soy el hijo\n");
    //Utilizamos la variable bandera para que el proceso no termine
    while (bandera);
}
else
{
    //proceso padre
    //Capturamos la señal SIGINT para luego enviar la señal al hijo
    signal(SIGINT, manejador);
    //Esperamos que termine el hijo
    wait(&status);
}
}

void manejador(int signum)
{
    printf("%d", pid);
    //Comparamos si el parametro es SIGUSR1
    if (strcmp(parametro, "SIGUSR1") == 0)
    {
        printf("\nmandando la seÑal SIGUSR1 a mi hijo\n");
        //Enviamos la seÑal al hijo
        kill(pid, 10);
    }
    else
    {
        printf("\nmandando la seÑal SIGUSR2 a mi hijo\n");
        //Enviamos la seÑal al hijo
        kill(pid, 12);
    }
}

void manejador2(int signum)
{
    //Mostramos la seÑal que fue enviada por el padre
    printf("Recibi la seÑal %d de mi padre\n", signum);
    bandera = 0;
}

```

Un pequeño detalle: en algunos sistemas Unix, cuando se recibe una seÑal y se ejecuta la funci3n, se restaura autom1ticamente la funci3n por defecto. Por ello, a veces es

necesario que la función termine volviendo a ponerse ella misma como tratamiento de esa señal.

También es posible hacer que una señal se ignore, es decir, que no la trate nadie, ni la función propia ni la de por defecto. Para ello hay que poner:

```
signal (numeroSeñal, SIG_IGN);
```

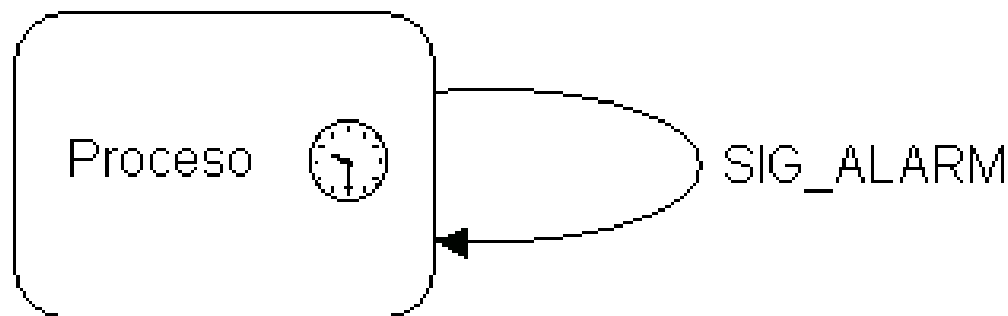
Para restaurar el tratamiento por defecto de la señal:

```
signal (numeroSeñal, SIG_DFL);
```

Una utilización bastante potente de las señales es el uso de SIGALARM para crear temporizadores en los programas. Con la función alarm() lo que se consigue es que el proceso se envíe a sí mismo una señal SIGALARM en el número de segundos especificados. El prototipo de alarm() es el siguiente:

```
unsignedintalarm(unsignedintseconds);
```

En su único parámetro se indican el número de segundos que se deseen esperar desde la llamada a alarm() para recibir la señal SIGALARM:



**La llamada a la función alarm() generará una señal SIG\_ALARM hacia el mismo proceso que la invoca.**

El valor devuelto es el número de segundos que quedaban en la anterior alarma antes de fijar esta nueva alarma. Esto es importante: sólo se dispone de un temporizador para usar con alarm(), por lo que si se llama seguidamente otra vez a alarm(), la alarma inicial será sobrescrita por la nueva.

A continuación, un ejemplo de su utilización:

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>

//Declaramos los prototipos de funciones
void manejador(int signum);
//Variable global
int bandera = 1;

//Función principal
main(int argc, char **argv)
{
    //Capturamos la señal SIGALRM
    signal(SIGALRM, manejador);
    printf("EN 10 segundos se creara una alarma\n");
    //Crear alarma en segundos
    alarm(10);

    //Mientras bandera sea 1, no finalizar el programa
    while (bandera)
        ;
}

//Funcion manejador
void manejador(int signum)
{
    printf("\nRecibi una alarma\n");
    //signal(SIGINT,SIG_DFL);
    bandera = 0;
}
```